AstroComms

AstroComms v1.9 / v2.x

User Manual v1.0

Requires ShadowMD (our version), AstroComms Firmware, BetterDuino (current version), and the sound firmware — alternatively MarcDuino.

Version 1.0 — 2026-04-21

Project repository:

https://github.com/PrintedDroid/ShadowMD-AstroComms-v1.-v2.x

AstroComms Ultra Shield — designed by walex and nitewing, 2020 (printed-droid.com)

Disclaimer and Liability Notice

AstroComms is a do-it-yourself project. Building, wiring, configuring and operating the system involves work that, if done incorrectly, can cause damage to property or persons.

By using this documentation, hardware, and software you acknowledge:

- You are solely responsible for all work on your system.
- You have the required basic knowledge or seek qualified help.
- You verify voltage, polarity, fuses, connectors before every power-on.
- You only use components matching the application.
- You comply with local regulations.

The author and all contributors accept no liability for damage caused by incorrect wiring, wrong components, improper operation, or insufficient safety measures.

The documentation and software are provided **as is** without warranty of any kind. If in doubt, seek qualified help before working on the system.

Table of Contents

Project repository (sketches, firmware updates, issues): **<https://github.com/PrintedDroid/ShadowMD-AstroComms-v1.-v2.x>**

Full user manual for AstroComms v1.9 / v2.x. Requires ShadowMD (our version), AstroComms Firmware, BetterDuino (current version), and the sound firmware — alternatively MarcDuino.

Contents

1. What is AstroComms

AstroComms is the central control board for an astromech droid (R2-D2 or variant). It carries all the controllers your droid needs for driving, dome rotation, panel animations, sounds, and app remote control.

What you can do with it:

- Drive and rotate the dome via PS3 Navigation Controller (Bluetooth)
- Drive and control via smartphone app (WiFi)
- Play sound effects and music (MP3 files from SD card)
- Open dome and body panels, move holos, animate the Magic Panel
- Use up to two PS3 controllers simultaneously (foot + dome)

This manual targets the current **v2.x** revision. If you are still using **v1.9**, the differences are documented separately in the "Connecting motors" section.

2. What you need

Already present on the AstroComms board (sockets):

- Arduino Mega 2560 with **AstroComms Firmware**
- Arduino Mega 2560 with this **ShadowMD version** — has the USB Host Shield mounted underneath
- ESP32, ESP8266, or ESP32-C3 Mini — one of these three modules
- Arduino Nano as **Body Master** with BetterDuino firmware (recommended) — MarcDuino possible as emergency fallback, but BetterDuino is significantly better maintained

- Arduino Pro Mini as **sound controller** — with `DFPlayer_Sound_control_v1.2` (recommended) or MarcDuino V3 in DFPlayer mode (alternative)
- DFPlayer Mini sound module (socketed on the board)

What you provide yourself:

- **Bluetooth dongle** — CSR8510-based. Recommended: LogiLink BT0015 (BT 4.0) or Asus USB-BT500 (BT 5.0). Also works: UGREEN CM748 (BT 5.3), LogiLink BT0048 (BT 4.0), generic CSR8510 dongles. **Not compatible**: TP-Link UB400 V2, LogiLink BT0058, most Realtek-based BT 5.0 dongles.
- **PS3 Navigation Controller** — 1 for driving, optional 2nd for dome
- **MicroSD card** for the DFPlayer (up to 32 GB, FAT32)
- **Speaker** (3–8 W) with amplifier
- **Motor driver** depending on setup: Sabertooth + SyRen (serial), VESC/BLDC ESC (PPM), or H-bridge (PWM+DIR, e.g. Cytron MD or BTS7960)
- **Power supply 12 V or 5 V** with enough capacity. **5 V recommended. Never feed both** at the same time.

On your computer:

- **Arduino IDE** 1.8.19 or 2.x (both tested) — https://www.arduino.cc/en/software
- **xLoader** (Windows) — to flash the `.hex` file for the AstroComms firmware
- USB cable for the Megas, USB-Serial adapter (FTDI or CH340) for the Pro Mini
- **VESC Tool** — only for **Mode 1** (brushless / VESC drives). The official VESC configuration software. **Mandatory**: each VESC/FSESC must be configured once with it, otherwise it does not respond to the droid (see section 3.2).

---

3. First-time setup

3.1 Plug modules into their sockets

Plug all modules in with the **board powered off**. Watch for correct orientation (Pin 1 marked with a silkscreen triangle).

- ShadowMD Mega with USB Host Shield underneath — into its dedicated socket
- AstroComms Mega into its socket
- ESP module — ESP32, ESP8266, or ESP32-C3 Mini, one on the 2×4 socket
- Nano (Body Master) — into the Nano socket next to the servo connectors
- Pro Mini (sound) — into its socket
- DFPlayer Mini — into the DFPlayer socket (with SD card inserted)
- Optional: Bluetooth dongle into the USB Host Shield

3.2 Connecting motors

Motor connections differ depending on board revision.

Motor modes at a glance

AstroComms supports three foot-drive modes. **Choose the mode by your motor type first**, then by the controller you own. The mode names refer to the *control signal*, not the motor type:

Mode	Motor type	Control signal	Example controllers	Signals per motor
Mode 0	Brushed DC	Packetized serial	Sabertooth 2x32 / 2x25	shared serial bus
Mode 1	**Brushless (BLDC)**	PPM / servo	VESC, FlipSky FSESC 6.7, standard RC ESC	1 signal wire
Mode 2	Brushed DC	PWM + direction	H-bridge: BTS7960, Cytron MD13S	2 wires (+ shared brake)

The key split:

- **Brushless motor** (3 phases) → needs an ESC/VESC → **Mode 1**
- **Brushed motor** (2 wires) → Sabertooth serial (**Mode 0**) *or* H-bridge like BTS7960/Cytron (**Mode 2**)

> ■■ *Only **Mode 1** drives brushless motors. **Mode 2 is also brushed** — an H-bridge cannot commutate a 3-phase brushless motor, despite the "BLDC header" silkscreen label on the board. The mode is set in the sketch via* `#define FOOT_CONTROLLER` *when flashing (see section 4).*
>
> ■■ ***Mode 1 requires one-time VESC Tool setup.** A VESC/FSESC does **not** respond to the droid out of the box — each controller must first be configured with the VESC Tool PC software (App = PPM, pulse 1.0–2.0 ms, etc.). Without this, the motors stay dead even if everything is wired correctly. Step-by-step instructions are in the FSESC example below.*

Shield v2 (current)

5-pin screw terminal for BLDC / PWM motor drivers:

Silkscreen label	Function
`PWM Left`	PWM speed, left foot motor
`DIR Left`	Direction, left foot motor
`BRK`	Shared brake (both motors)
`PWM Right`	PWM speed, right foot motor
`DIR Right`	Direction, right foot motor

Also available: a 2-pin header carrying `PWM Left` and `PWM Right` — useful for PPM/servo ESCs like VESC that need only one signal wire per motor.

The brake line is only driven in Mode 2 (PWM+DIR). In Mode 1 (PPM) it stays idle.

Shield v1.9 (older revision)

The 8-pin header exposes Mega pins in this order: D19, D17, D32, D11, D8, D7, D6, D5. All 8 pins are **free to use** — they are not hardwired to the serial buses. D19 / D17 / D32 are digital only (suitable for DIR / brake), pins D5–D11 also provide PWM.

Silkscreen note: D19 and D17 are labeled `RX1` and `RX2` on v1.9. They were originally intended as inputs for two Sbus / iBus RC receivers (e.g. for a HOTRC DS600 radio transmitter). The Shadow firmware does not currently implement this option — so these pins can alternatively be used as free digital I/O for BLDC DIR or brake.

Recommended assignment (matches the `ASTROCOMMS_SHIELD_V19` sketch preset):

Function	Pin used
PWM Left	D6
DIR Left	D32
PWM Right	D7
DIR Right	D8
BRK	D5

D11 remains free as a reserve for future extensions.

> ***Brushless / Mode 1 on v1.9 uses only D6 and D7.*** *The table above is the full* ***Mode 2*** *(brushed, PWM+DIR) assignment. For a brushless drive (VESC/FSESC), the two PWM pins D6 / D7 carry the PPM signal — one wire per motor — and DIR (D32 / D8) and BRK (D5) stay unused.*

Additionally, v1.9 has a **3-pin Cytron terminal** (labels `GND` / `DIR` / `PWM`). It uses the free Mega pins D3 and D4. Originally intended as an alternative to the SyRen10 for the dome motor. The Shadow firmware does not currently support this mode — the terminal is unused on v1.9 for now.

Mode 0 (Sabertooth serial)

Does not use any of the BLDC pins. Connection is via the 6-pin communication terminal (see 3.3) to the Sabertooth's S1 input. Sabertooth and SyRen share the line and are distinguished by Packetized Serial address (128 / 129).

Example: 2× FlipSky FSESC 6.7 (or VESC) in Mode 1

If you're building a BLDC drive with two VESC-based controllers (FSESC 6.7, BenjaminFB VESC, Trampa VESC etc.), you use **Mode 1 (PPM/Servo)**. Each motor gets **one** signal wire. Brake and direction are NOT driven from the shield — the VESC handles both internally.

Sketch configuration:

```
// Set the shield revision to match your board – uncomment EXACTLY one
// (leaving both active triggers a compile #error):
#define ASTROCOMMS_SHIELD_V2      // current board
//#define ASTROCOMMS_SHIELD_V19   // older board (uncomment this instead)

// Mode 1 = PPM/VESC:
#define FOOT_CONTROLLER 1
```

Pin assignment (automatic, based on shield revision):

Signal	Shield v2	Shield v1.9
`leftFootPin`	D44 (PWM Left on the 5-pin terminal or 2-pin header)	D6 (on the 8-pin BLDC header)
`rightFootPin`	D45 (PWM Right)	D7 (on the 8-pin BLDC header)

Wiring per FSESC (both controllers identical):

FSESC pin	To shield
Signal (signal wire, usually white or yellow)	`leftFootPin` or `rightFootPin`

FSESC pin	To shield
GND (black)	GND pin on the 6-pin communication terminal (or any other GND on the shield)
+5V (red)	**DO NOT connect** — the FSESC has its own supply

Power the FSESC **separately** from the drive battery. Only GND is shared with the shield.

VESC Tool configuration — MANDATORY (per FSESC, one-time). Without this, the VESC ignores the PPM signal from the shield and the droid won't move, no matter how it's wired:

1. Run the Motor Setup Wizard (inductance/resistance, hall or sensorless).
2. App Settings → General → App to Use: **PPM** (not "PPM and UART", not "Nunchuk").
3. App Settings → PPM: - Control Type: **Current** — the correct choice for a droid because: - **Reverse is enabled**: stick back (<1500 µs) = motor runs in reverse. - **Brake is enabled**: if the stick is pushed opposite to the current direction of motion, the VESC brakes actively with negative current (regenerative — energy returns to the battery). Stick at center (1500 µs) also brakes a free-spinning motor. - Alternatives like `Current No Reverse` or `Current No Reverse With Brake` disable parts of this — for a droid that needs both reverse and braking, `Current` **is the only right choice**. - Pulse Length Start: `1.0 ms` - Pulse Length End: `2.0 ms` - Pulse Length Center: `1.5 ms` - Deadband: 15–20 % - **Enable Safe Start** — prevents motor startup on reconnect mid-ride.
4. Motor Settings → Current → set **Motor Current Max Brake** (e.g. 30–60 A negative, depending on FSESC current budget). Without this limit the brake in `Current` mode is theoretically unbounded — realistic values make brake behavior controlled and protect the battery.
5. "Write Configuration" and "Write to Flash".

Adjusting direction: use the Shadow CLI with `invert left on` / `invert right on` , then `invert save` . Survives reboot. See section 10.

Unused BLDC pins: all other BLDC header pins remain free (v1.9: D5, D8, D11, D17, D19, D32; v2: 42, 43, 40 from the 5-pin terminal). No harm.

Dome drive: unaffected. SyRen 10 on Serial2 continues to drive the dome regardless of foot drive mode.

> ■■ *FSESCs with high current settings accelerate significantly more aggressively than Sabertooth. **Dry-test is mandatory** before the droid touches the floor for the first time — see section 12.*

3.3 Outer connectors

Servo strip next to the Nano

12 servo connectors numbered **1 to 12**. Connects panels / arms / doors of the droid to the BetterDuino firmware.

Separate servo power supply via the dedicated voltage terminal. Do not power servos from the system 5 V — high current spikes when several servos move simultaneously disturb the electronics.

Which servo goes to which number depends on the BetterDuino role of the Nano:

Body Master (standard on this board):

Servo #	Function
1	Data Panel Left (DPL)
2	Utility Arm Upper

Servo #	Function
3	Utility Arm Lower
4	Left Body Door
5	Left Arm
6	Left Arm Tool
7	Right Body Door
8	Right Arm
9	Right Arm Tool
10	Charge Bay Door
11	under development
12	under development

Dome Master (if the Nano runs as Dome Master): Servo 1–10 are dome panels 1–10. Servo 11 and 12 are under development.

Dome Slave (holoprojectors): Servo 1/2 = Front Vertical/Horizontal, Servo 3/4 = Rear V/H, Servo 5/6 = Top V/H, Servo 7–9 = HP LEDs (Front/Rear/Top), Servo 10 = extra channel, Servo 11 and 12 under development.

Servos 11 and 12 are still being developed — do not connect servos there yet.

Audio Line Out

Right of the Nano: **3.5 mm jack** and below it a **3-pin header** — both carry the **same Line Out signal** from the DFPlayer. Connect to an external amplifier. Only one of the two needs to be used (two physical connector options).

Speaker test terminal (2-pin)

For a passive speaker directly on the DFPlayer — **for functional test only**. In the finished droid, sound should go via Line Out into an amplifier.

6-pin communication terminal (bottom left)

Pin	Signal	Meaning
1	GND	Ground
2	Motors	Sabertooth / SyRen serial bus (both on the same pin, distinguished by address 128 vs. 129)
3	Aux	Optional signal channel (currently no function, for future extensions)
4	MP3	MP3 command signal for external sound output
5	Flthy TX	FLTHY HP controller commands
6	Dome TX	Dome MarcDuino / BetterDuino commands

RJ45 to dome / slipring

Carries Aux, MP3, Flthy TX, and Dome TX through the slipring board into the dome. Standard RJ45 wiring.

Slave header (right next to the Nano)

Optional socket for a second Nano with BetterDuino in slave role — e.g. for additional servos or panel extensions.

RC IN (pin header)

Currently unused. Reserved for future RC receiver integration.

Note: on **v1.9**, the pins labeled `RX1` / `RX2` on the BLDC header (D19 / D17) are an alternative entry point for Sbus / iBus RC receivers — e.g. for two HOTRC DS600 receivers. The Shadow firmware has no logic for this yet, but the option is open for future extensions.

3.4 Jumpers and DIP switches

ESP power (U76 / U77): set one of the two jumpers, depending on your ESP module.

- `U76` = 5 V to ESP (e.g. ESP32 DevKit)
- `U77` = 3.3 V to ESP (e.g. ESP8266)

ESP RX/TX assignment (below the ESP socket): some ESP variants have swapped RX/TX pinouts. These jumpers let you correct the UART assignment. If the ESP doesn't receive or transmit anything after first flashing, check these jumpers.

PWR Jumper 1: can technically bridge system voltage to the servo supply — **not recommended**. Always power servos separately.

MP3 jumper: disconnects the MP3 line toward the dome (via RJ45). Background: in classic MarcDuino setups the Dome Master sends sound commands to the sound module. On this board the Pro Mini in the body handles sound generation; the path to the dome is not needed. Leave the jumper in default position unless you're running a hybrid setup.

ISP jumper: must be **off** when flashing the Pro Mini via ISP (only needed with MarcDuino V3). The Pro Mini's ISP pins sit **underneath the DFPlayer module** — remove the DFPlayer first for access.

DIP switch (3-position):

- Switches **1 and 2**: interrupt serial communication to the Nano. Before flashing the Nano, turn both off; turn them back on afterwards.
- Switch **3**: toggles the status LEDs on/off.

---

4. Flashing firmware

4.1 ShadowMD via Arduino IDE

1. Open **Arduino IDE**.
2. Open sketch folder: `Shadow_MD_AstroCan_BLDC_Standalone_v2/` → the `.ino` loads automatically.
3. Set two things at the top of the sketch: - **Shield revision**: `` `#define ASTROCOMMS_SHIELD_V2 // current board (default) //#define ASTROCOMMS_SHIELD_V19 // older board` `` - **Motor mode**: `` `#define FOOT_CONTROLLER 0` `` - 0 = brushed DC, Sabertooth serial - 1 = brushless ESC/VESC, PPM (e.g. FlipSky FSESC 6.7) - 2` = brushed DC, PWM+DIR via H-bridge (Cytron MD-Series, BTS7960)
4. **Board**: `Tools → Board → Arduino AVR → Arduino Mega or Mega 2560` .
5. **Port**: select the ShadowMD Mega's COM port.
6. **Upload** (Ctrl-U or arrow button).

On success: "Done uploading." in the bottom pane.

> ■■ ***IMPORTANT — Dry-test before first drive****: after first flashing,* ***always block up the droid*** *(wheels off the floor) before the motors get powered. Wrong motor direction, incorrect pin mapping or unexpected ramping values can otherwise cause the droid to run away uncontrolled. Only put it back on the ground once everything reacts cleanly — direction, steering, stop. Details in safety chapter 12.*

4.2 AstroComms firmware via xLoader

The AstroComms firmware is flashed as a prebuilt `.hex` file — no source code needed.

1. Launch **xLoader**.
2. **Hex file**: select `AstroCommsV112.hex` .
3. **Device**: `Mega(ATMEGA2560)` .
4. **COM port**: the AstroComms Mega's port (not the ShadowMD!).
5. **Baud rate**: `115200` .
6. Click **Upload**.

xLoader shows progress + success at the bottom. Takes 10–30 seconds.

4.3 ESP module via Arduino IDE

Only needed if you want to use the app.

1. **Install board support**: - `File → Preferences → Additional Boards Manager URLs` : `https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json` - `Tools → Board → Boards Manager` → search "esp32" → install.
2. Open sketch: `esp32_esp8266_as_xbee_v2.0/esp32_esp8266_as_xbee_v2.0.ino` .
3. If desired: change WiFi SSID and password (default: SSID `R2D2 Astromech` , password `C3POsucks`). **Strong password strongly recommended** if the droid appears in public.

4. Select board (e.g. `ESP32 Dev Module` or `NodeMCU 1.0` depending on module).
5. Upload Speed: `921600` .
6. Select port and upload.

If you don't want to use ESP: leave the socket empty. The rest of the system works without it.

4.4 Nano — BetterDuino (pre-flashed)

BetterDuino Firmware V4 (author: RealNobser, upstream: https://github.com/RealNobser/BetterDuinoFirmwareV4) replaces the old MarcDuino firmware with a modern, actively maintained C++ rewrite.

Features:

- Single firmware covers all roles: **Dome Master**, **Dome Slave**, **Body Master**. Auto-configures on first boot; switch manually via `#MD00` / `#MD01` / `#MD02` if needed.
- All settings in EEPROM: servo min/max, panel sequences, sound backend, runtime tuning
- Four sound hardware types supported: MP3 Trigger, DF Mini Player, R2-D2 Vocalizer, DY-Player
- Dome Lift integration
- Community support: Facebook "Printed Droid" group, Astromech.net forum

The Nano ships **pre-flashed with BetterDuino in Body Master role** on this board. The end user does not need to do anything here. Configuration is done via serial commands (see section 9).

4.5 Pro Mini — sound controller

Two options. **Option A recommended — significantly simpler, no special hardware needed.**

Option A (recommended): `DFPlayer_Sound_control_v1.2` via FTDI

Arduino IDE workflow, USB-Serial adapter (FTDI / CH340) is enough:

1. Connect USB-Serial adapter to the Pro Mini's serial header (GND, 5V, adapter TX to Pro Mini RX, adapter RX to Pro Mini TX, DTR to Reset).
2. Arduino IDE: open sketch `DFPlayer_Sound_control_v1.2/DFPlayer_Sound_control_v1.2.ino` .
3. If needed: adjust `bank_max_sounds[]` (array at the top) to the actual MP3 file counts.
4. Board: `Arduino Pro or Pro Mini` , processor: `ATmega328P (5V, 16 MHz)` .
5. Port: USB-Serial adapter's COM port.
6. Upload.

Option B (fallback): MarcDuino V3 via ISP

Only use this if an existing MarcDuino V3 installation should be kept. Requires an ISP programmer (e.g. USBasp, AVR Dragon, Arduino-as-ISP), knowledge of avrdude, and setting fuses.

1. **Turn off** the ISP jumper on the board.
2. Remove the DFPlayer module to access the Pro Mini's ISP pins (they sit underneath).
3. Connect ISP programmer to the ISP pins.
4. Set fuses: `L=0xFF` , `H=0xD7` , `E=0xFC` .
5. Flash the MarcDuino V3 `.hex` .
6. Disconnect ISP, put the DFPlayer back, turn ISP jumper back on.
7. Send `#SM01` via serial to put the MarcDuino in DF Mini Player mode.

Comparison of the two options:

	Option A (DFPlayer_Sound_control_v1.2)	Option B (MarcDuino V3)
Flashing	Arduino IDE + USB-Serial adapter	ISP programmer + avrdude
Setting fuses	not needed	required (`L=FF / H=D7 / E=FC`)
Configuration	ready	`#SM01` to switch into DFPlayer mode
Sketch editing	yes (Arduino IDE)	only C source + Makefile
Effort for tweaks	open Arduino IDE, upload	full toolchain setup

Bottom line: Option A has less effort and fewer failure modes. Option B only makes sense if a MarcDuino setup already exists and should not be changed.

---

5. Preparing the SD card

One single layout for **both** sound options (DFPlayer_Sound_control_v1.2 **and** MarcDuino V3 in DFPlayer mode):

1. Format the SD card as **FAT32**.
2. Create a folder named `mp3` in the root.
3. Put MP3 files into the `mp3` folder, each with a **4-digit number at the start of the filename**, followed by any suffix after an underscore:

```
mp3/
  0001_gen-1.mp3
  0002_gen-2.mp3
  ...
  0042_chat-17.mp3
  ...
  0151_leia-1.mp3
  ...
  0254_Startsound.mp3
  0255_Startsound.mp3
```

The DFPlayer identifies files by the **first four digits**. Everything after the underscore is free-form description — pick whatever helps you.

Bank convention (MarcDuino standard):

Bank	Files	Typical content
1	0001–0025	general sounds
2	0026–0050	chat / beeps
3	0051–0075	happy
4	0076–0100	sad
5	0101–0125	whistles
6	0126–0150	scream

Bank	Files	Typical content
7	0151–0175	Leia / messages
8	0176–0200	music / cantina / themes
9	0201–0225	reserved
—	0254, 0255	startup sounds (MarcDuino convention)

Not every bank has to be full. If you use Option A, you can adjust `bank_max_sounds[]` in the sketch to match the actual counts.

Default volume: middle (about 15 of 30). Changeable at runtime via the PS3 controller or CLI (see section 9).

---

6. Pairing PS3 controllers

The system needs to learn your PS3 controllers' MAC addresses once. Done via serial console.

6.1 Preparation

- Fully charged PS3 Navigation controllers
- USB cable to the controller (mini-USB)
- ShadowMD Mega plugged in and connected via USB
- Bluetooth dongle **not yet** plugged in

6.2 Pairing procedure

1. Arduino IDE → **Tools** → **Serial Monitor**. Set baud rate to **115200** in the bottom right.
2. Type `pair` and press Enter.
3. The console reports which slot is active: foot → dome → foot spare → dome spare.
4. **For each controller**: - Unplug the BT dongle (if present). - Connect the PS3 controller via USB cable directly to the USB Host Shield on the ShadowMD Mega. - Console reports "Paired [slot]: XX:XX:XX:XX:XX:XX". - Unplug the controller. - Plug the BT dongle back in.
5. With two controllers, repeat. Spare slots (3 and 4) are optional.
6. When done: type `done` and press Enter.
7. `status` shows the stored MACs for verification.

If a pairing attempt fails: `clear` erases all entries, then start over with `pair` .

6.3 Connecting controllers

After successful pairing:

1. Restart the Mega (reset button).
2. Plug in the BT dongle.
3. Press the **PS button** on the PS3 controller.
4. After about 5 seconds, one of the four controller LEDs lights solid → connected.

The first controller is the foot controller; the second (if connected) takes over the dome. **The second controller typically takes longer to connect than the first** — be patient.

6.4 Turning controllers off

- **Hold the PS button for 15 seconds**, or
- **PS + L3** simultaneously

7. Driving and operating

PS3 foot controller

- **Analog stick**: forward / back / left / right — drive + steering (BHD diamond mixing)
- **L1 + stick** or **L2 + stick** (depending on config): dome rotation via the foot controller if no second controller is connected
- **D-pad + Triangle / Square / Circle / X**: configurable shortcuts for MarcDuino sequences, sounds, panels
- **PS button long press**: disconnect

PS3 dome controller (optional)

- **Analog stick**: dome rotation
- Own button shortcuts for dome panels, holos, sounds

Automatic dome movements

When enabled, the dome rotates randomly during idle periods. Toggleable via a button shortcut (configured in the sketch).

Sound shortcuts

Quick to trigger via PS3 buttons:

Shortcut	Sound
`$L`	Leia message
`$C`	Cantina (orchestra)
`$c`	Cantina beep (R2 version)
`$S`	Scream
`$F`	Faint / short circuit
`$D`	Disco
`$W`	Star Wars theme
`$M`	Imperial March

8. Using the app over WiFi

If the ESP is installed and flashed:

1. Power on the droid.

2. On your phone/tablet: select WiFi `R2D2 Astromech` , enter password `C3POsucks` (or the one you set).
3. Open your R2 control app, choose **TCP** as connection type.
4. **Host**: `192.168.4.1`
5. **Port**: `9750`
6. Connect.

Up to **4 devices** can be connected simultaneously (e.g. your phone + tablet + a friend's button board). The AstroComms firmware queues commands FIFO to avoid collisions.

The app can send all standard MarcDuino commands.

XBee note: the ESP socket can theoretically also be populated with a classic XBee module instead of an ESP. In practice, this board uses the ESP path — simpler, faster, and every current R2 app supports TCP.

---

9. Command reference

9.1 ShadowMD CLI (Serial Monitor, 115200 baud, at the ShadowMD Mega)

Command	Meaning
`help`	List of all commands
`status`	Current status: paired controllers, connection, invert settings
`pair`	Enter PS3 pairing mode
`done`	Exit pairing mode
`clear`	Delete all paired controllers
`invert`	Show motor direction settings
`invert left on` / `off`	Invert or normalize the left side
`invert right on` / `off`	Invert or normalize the right side
`invert save`	Save invert settings persistently
`invert reset`	Set both sides to normal and save
`debug`	Toggle verbose Bluetooth debug output
`reboot`	Software reset

9.2 Sound and panel commands (via app / WiFi / XBee)

All commands end with Carriage Return (`\r`).

Prefixes:

- `:` = dome command (e.g. `:SE01` = dome panel sequence 01)
- `;` = body command
- `$` = sound
- `+` = FLTHY HP direct

- `*` = broadcast dome + FLTHY
- `_` = body config

Common commands:

Command	Effect
`:SE01`	Dome sequence 1 (Close All Panels)
`:OP00`	Open all dome panels
`$R`	Random sound mode on
`$O`	Mute
`$s`	Stop sound
`$+` / `$-`	Volume up / down
`$m` / `$f` / `$p`	Volume mid / max / min
`$L` , `$C` , `$c` , `$S` , `$F` , `$D` , `$W` , `$M`	Sound shortcuts (see table above)

9.3 MarcDuino V3 config commands (only for Option B)

At the Pro Mini's Serial Monitor (9600 baud), or via the body bus:

Command	Effect
`#SM00`	Sound backend MP3 Trigger (default)
`#SM01`	Sound backend DF Mini Player
`#SQ00` / `#SQ01`	Chatty mode on / off
`#SS00` – `#SS03`	Startup sound off / default / alt 1 / alt 2
`#SD00` / `#SD01`	All servo directions normal / reversed
`#SRxxy`	Set a single servo direction (xx=number, y=0/1)

10. Adjusting motor direction

> ■■ ***Block up the droid first!*** *Before testing invert settings, put the droid on blocks (wheels free). If* `invertLeft` *is wrong-way-around, the left motor runs backward on forward stick — the droid takes off sideways uncontrolled. Only put it back on the ground once the direction reacts cleanly.*

If the driving direction is reversed on first test (e.g. left motor turns the wrong way):

1. Open the Serial Monitor at the ShadowMD Mega (115200 baud).
2. `invert left on` → Enter. Test.
3. If the right is also reversed: `invert right on` .
4. If all good: `invert save` → setting survives the next boot.

Return to defaults: `invert reset` .

Note: only effective in BLDC modes (FOOT_CONTROLLER 1 and 2). In Sabertooth mode there is a separate inversion that is changed in the sketch source.

11. Troubleshooting

Sketch won't compile

- `#error "No shield revision defined"` : one of the two shield lines at the top of the sketch must be active (`ASTROCOMMS_SHIELD_V2` or `ASTROCOMMS_SHIELD_V19`).
- `#error "Define only ONE shield revision"` : both lines active — comment one out.
- **USB Host Shield library conflict**: the `src/` folder in the sketch directory must stay intact. Don't overwrite it with an Arduino Library Manager version.

Controller won't connect

- BT dongle compatible? LogiLink BT0015 and Asus USB-BT500 are recommended; TP-Link UB400 V2 and Realtek BT 5.0 dongles typically do not work.
- Pairing done? Check with `status` in the Serial Monitor — if "XX" instead of a MAC, the slot is empty.
- After `pair` , don't forget to send `done` before trying to connect.
- Restart the Mega, then press PS.
- The second controller takes longer to connect than the first — not unusual.

No sound

- SD card FAT32? `mp3/` folder present? Files with 4-digit prefix?
- DFPlayer module properly socketed?
- Speaker on Line Out (jack or 3-pin header) via amplifier — not permanently on the 2-pin speaker test terminal?
- Attach a USB-Serial adapter to the Pro Mini, open monitor at 9600 baud, send `$101` → should play file `0001_...mp3` .
- If the Pro Mini runs MarcDuino V3: send `#SM01` to activate DFPlayer mode (default is MP3 Trigger).

Driving direction wrong

- `invert left on` / `invert right on` , `invert save` — see section 10.

WiFi not visible

- ESP module plugged in? Power jumper set correctly (U76 = 5 V, U77 = 3.3 V)?
- RX/TX jumpers below the ESP correct? Some ESP modules have swapped pinouts.
- Connect the ESP to the PC via USB, open Serial Monitor at 9600 baud → "Access Point started successfully" visible?
- Smartphone on the 2.4 GHz band? ESP32/ESP8266 only support 2.4 GHz.

Motor stutters / runs unevenly

- Power supply too weak? Current spikes on acceleration need proper capacity.
- Loose screw/plug connection at the motor terminal or controller? Retighten all.
- BLDC Mode 2 without brake wired? Connect the brake pin or comment out `ENABLE_BRAKE` in the sketch.

App connects but nothing happens

- Port actually **9750**?
- App connection type = **TCP**, not XBee API.
- Telnet test from PC: `telnet 192.168.4.1 9750` — should be open.
- ESP firmware flashed?

Pro Mini won't flash (MarcDuino path)

- ISP jumper off?
- DFPlayer module removed to access the ISP pins?
- Correct fuses set (`L=0xFF / H=0xD7 / E=0xFC`)?
- Alternatively switch to Option A (DFPlayer_Sound_control_v1.2) via FTDI — significantly simpler.

Nano (BetterDuino) can't be reached via serial

- DIP switches 1 and 2 on? (When they're off, serial communication to the Nano is disconnected.)

Status LEDs off

- Turn on DIP switch 3.

Servos twitch / overloaded

- Servo power supply connected separately, PWR Jumper 1 in default position (not bridged)?
- Servo power supply delivers enough current for all servos simultaneously?

Servo 11 or 12 won't move

- These two channels are currently **under development** — don't connect servos there yet.

Dome doesn't rotate although foot motors work

- SyRen DIP switches: Packetized Serial mode (1, 2 and 4 down, others up)?
- SyRen motor M1/M2 terminals swapped?
- Address 129 set correctly on the SyRen board?

Dome rotates uncontrollably fast or jerks

- Reduce `domespeed` in the sketch (default is 100, range 0-127).
- Check ramping parameters — too aggressive ramping causes jerking.

Two controllers don't connect simultaneously

- CSR4-based dongles often hold only **one** controller at a time — that's a dongle limitation, not a firmware problem.
- Solution: use a CSR8510-based dongle (LogiLink BT0015, Asus USB-BT500 — see section 2).
- Alternative: connect the second controller only after ~10 s once the first is stable.

Sound cuts out / crackles

- Voltage dip at the DFPlayer — use separate audio supply or add buffer capacitor at the module.
- Use SD card class 10 or higher, known brands (SanDisk, Samsung) — cheap fake cards are a common cause.

- Reformat SD card (FAT32), copy files fresh without fragmentation.

Compile error "xxx.h: No such file or directory"

- Bundled `src/` folder in the sketch is missing or got overwritten by Arduino Library Manager.
- Re-fetch the sketch from Git release/zip, the `src/` folder **must** stay.

After firmware update: `status` shows "XX" instead of MAC

- EEPROM magic byte no longer matches — happens after `clear` , first boot of a new Mega, or EEPROM corruption.
- Just run `pair` again, see section 6.

BT dongle gets hot / connection drops constantly

- Quality/power issue with the dongle. Try a different one from the recommended list (section 2).
- Keep USB cable short, use high-speed cable.
- Distance from interfering sources (WiFi router, microwave, other BT devices).

App command doesn't reach the droid

- ESP firmware actually flashed? On ESP USB-Serial monitor (9600 baud), look for "Access Point started successfully".
- No XBee simultaneously in the socket? (ESP and XBee share the same UART and exclude each other.)
- App connection type **TCP** (not XBee API mode).
- Test: connect USB debug to the AstroComms Mega, Serial Monitor (9600 baud) — commands from the ESP should show up there.

Droid reacts with delay to stick input

- BT interference from nearby WiFi router, microwave or other BT devices.
- Dongle with external antenna outperforms tiny stubby dongles.
- Keep distance and line of sight to the droid.

Servo twitches at rest

- Many cheap servos oscillate when kept powered.
- BetterDuino has a servo-detach mechanism — configurable via serial command (see BetterDuino repo).
- Better servo models (digital, metal gears) twitch much less.

---

12. Safety

Dry-test at first boot (IMPORTANT)

Before the first drive always do a dry run:

- **Block up** the droid (wooden block, crate, stands) with the wheels free in the air.
- Plug the ShadowMD Mega via USB, keep motor power disconnected.
- Connect controller, gently move the foot stick → check **which direction** each side turns.

- If wrong: `invert left on` / `invert right on` / `invert save` (see section 10).
- Check dome rotation and panel movements idle first as well.
- **Only put the droid back on the ground** once direction, steering, ramping, and emergency stop all react cleanly.

Especially important for BLDC setups — PWM curves, brake behavior and acceleration are hard to predict without load. A droid shooting off unbraked on its first drive is a real injury risk.

Emergency stop

Pulling the BT dongle during driving stops the motor connection (timeout). An **additional hardware emergency off switch** in the power supply is mandatory at live events. The switch must be reachable by the driver at any time.

When flashing

Before every firmware upload: **disconnect motor power** (foot + dome). A half-uploaded sketch can produce unexpected signals on the motor pins.

Power supply

Feed 12 V **or** 5 V — **never both at once**. **5 V recommended.** Check for overvoltage protection and sufficient capacity.

Pairing hygiene

After conventions: reset spare slots (3 and 4) via `clear` so foreign controllers can't accidentally connect.

WiFi password

The default password `C3POsucks` is **public knowledge**. For events, definitely change it (section 4.3) — access to the WiFi means full access to the droid.

Separate servo power

Power servos **always separately**. Leave PWR Jumper 1 in default position. High current spikes when several servos move simultaneously can otherwise cause reset events or USB drops.

Pre-power-on check

Before the first power-on after modifications or new components:

- Check polarity of battery leads with a multimeter (plus to red, minus to black)
- No shorts between power and GND (resistance test with power off)
- Appropriate fuse in the main line — sized to cable cross-section and motor load
- All screw terminals tightened, no stray strands that could come loose

LiPo battery safety

- **Balance-charge** with a proper charger; never use swollen or puffed cells.
- Use a **LiPo safe bag** or fire-resistant container for storing and charging.
- **Disconnect during transport** — loose contacts during transport can short.
- Never overcharge, never deep-discharge, never leave in direct sunlight.

Shared GND with separate supplies

If motors and logic have separate power supplies: **tie the GND leads of both supplies together**. Without a shared ground, the potentials drift and control signals can break or trigger unexpected behavior.

Motor driver thermals

Sabertooth, SyRen, BLDC ESCs and H-bridge drivers get **warm** during operation. That's fine in normal use — on overheat, power limits kick in automatically.

- Ensure airflow inside the droid; no fabric/foam covering directly on the driver.
- During long driving sessions (conventions), pause occasionally and check temperature by hand.
- Passive heatsinks or small fans are recommended for powerful motors.

ESD (electrostatic discharge)

In dry weather or on carpeted floors:

- Before handling MCU modules, ESP, Nano, Pro Mini, touch a grounded metal object first (heating pipe, metal enclosure).
- Never grab modules by their pins — only by the edges.
- Avoid nylon/synthetic clothing when swapping modules.

Event / convention safety

At public appearances:

- **Keep visual contact** with the droid — never drive blind.
- **Safety distance** from children, pets, photographers — the droid has no sensors and won't dodge anything.
- **Emergency-off within reach** of the driver.
- **Battery reserve** — flat LiPos can drop out abruptly under motor load, and then the droid reacts unpredictably.
- **Don't drive while charging** — it's like mowing the lawn with an open plug.

Have fun with your droid!

Further information and community support: **printed-droid.com**, **astromech.net**, Facebook group "Printed Droid".